

RentIt

Del hvad du har. Lej hvad du behøver.

Platform til privat udlejning af genstande.

Produktrapport

Titelblad

Projekt titel: RentIt

Projektperiode: 23.03.2026 - 29.04.2026

Skole: Technical Education Copenhagen (TEC) EUX

Uddannelse: Data og Kommunikation Uddannelse

Hovedvejleder: Flemming Sørensen

Hold: 1205hf26046xp

Deltagere: Chanakya Joshi, Subarna Gurung og Marko Dankovich

INDHOLD

Læsevejledning.....	1
Indledning.....	1
Baggrund.....	1
Kravspecifikation.....	2
Oprindelige krav.....	2
Ændringer og udvidelser.....	2
Nye funktioner tilføjet undervejs.....	4
Definition af produktet.....	5
Funktionalitet.....	5
Brugerhåndtering.....	5
Opslag og udlejning.....	6
Browse og søgning.....	6
Favoritter og senest set.....	6
Lejeforespørgsel og lånehåndtering.....	6
Automatiske baggrundstjenester.....	6
Chat og beskeder.....	6
Anmeldelser.....	6
Tvister og konfliktløsning.....	7
Bødesystem.....	7
Rapportering.....	7
Verifikation.....	7
Support.....	7
Notifikationer.....	7
Dashboard.....	7
Adminpanel.....	7
QR-kode bekræftelse og snapshot.....	8
Kortvisning.....	8
Begrænsning.....	8
Betaling.....	8
Forsikring.....	8
Levering og fragt.....	8
Rate limiting.....	8
Email og push notifikationer.....	9
Fremtid - mulige udvidelser.....	9
Testkonditioner.....	9
Udviklingsmiljø.....	9

Unit tests - Service-laget.....	9
Unit tests - Repository-laget.....	9
Unit tests - Controller-laget.....	11
Brugertest.....	11
Brugervejledning.....	11
Teknisk produktdokumentation.....	12
Systemsarkitektur.....	12
Aktør kontekst diagram.....	12
Aktør beskrivelse.....	13
Bruger.....	13
Administrator (Admin).....	13
Developers.....	14
Frontend.....	14
Backend.....	14
Database.....	14
Docker Compose.....	14
SignalR og realtidskommunikation.....	15
Billedopbevaring.....	15
Kodestruktur.....	15
Kommunikationsflow.....	15
Sikkerhed.....	15
Ikke-funktionelle Krav.....	16
Hardware/Software miljø.....	17
Teknologier.....	17
Forkortelse og begreber.....	18
Bilagsoversigt.....	19

Læsevejledning

Denne rapport er produktrapporten for RentIt, som er et skoleprojekt udviklet af tre elever på TEC EUX Data og Kommunikation.

Rapporten beskriver det produkt vi har bygget, en webapplikation hvor private kan leje og udleje genstande til hinanden. Du vil i rapporten finde en beskrivelse af hvad systemet kan, hvad det ikke kan, og hvordan vi har testet det.

Rapporten er bygget op så du kan læse den fra start til slut, men du kan også hoppe direkte til det afsnit du er interesseret i ved hjælp af indholdsfortegnelsen ovenfor.

Den tekniske dokumentation og bilag findes bagerst i rapporten, og indeholder diagrammer, kode og testresultater.

Indledning

Dette dokument er produktrapporten for RentIt, en webapplikation udviklet som vores afsluttende projekt på TEC EUX Data og Kommunikation. Rapporten beskriver, hvad vi har bygget, hvorfor vi byggede det, og hvordan det fungerer teknisk.

Ideen bag RentIt opstod fordi vi så et problem i hverdagen, mange mennesker har ting stående derhjemme som de ikke bruger ret tit, mens andre har brug for præcis de samme ting men ikke vil købe dem til fuld pris for at bruge dem en gang. Der findes allerede udlejningsvirksomheder, der prøver at løse dette problem, men de har ofte et begrænset udvalg, høje gebyrer og er styret af en central virksomhed.

Vores løsning er en platform hvor private kan leje og udleje ting direkte til hinanden, uden mellemmand og med frihed til selv at bestemme pris og vilkår. Det kan være alt fra værktøj og havemaskiner til bøger, spil osv. Systemet er bygget så det er nemt og trygt at bruge, med et admin-system der sikrer at indholdet på platformen er gyldigt.

Rapporten er struktureret, så den først beskriver baggrunden for projektet, derefter hvad systemet kan og ikke kan, og til sidst den tekniske dokumentation og testresultater.

Baggrund

Der findes mange genstande, særligt værktøj og maskiner, som folk kun bruger få gange om året. Det kan fx være en plæneklipper, boremaskine, højtryksrenser eller en stige. For mange giver det ikke mening at købe dyrt udstyr, når behovet er sjældent. Samtidig har mange ting stående som bare samler støv, fordi de ikke ved hvem de kan låne dem ud til, eller fordi de ikke har en nem og tryk måde at gøre det på.

De eksisterende løsninger på markedet er typisk private udlejningsvirksomheder med et begrænset udvalg og høje gebyrer. Alternativet er at skrive i en Facebook-gruppe, men det er

ustruktureret, uoverskueligt og giver ingen sikkerhed for hverken udlejer eller lejer. Der er simpelthen ikke en god løsning til private, der vil leje ting af hinanden på en nem og tryk måde.

Derfor valgte vi at bygge RentIt, en platform der giver begge parter det, de har brug for. Udlejeren kan tjene lidt penge på ting der ellers bare står og samler støv, og lejeren kan spare penge ved kun at betale for den periode hvor tingene faktisk bruges. Med en fast pris pr. dag, et depositum som sikkerhed og et chat-system til at aftale detaljer, er det meget mere struktureret og trykt end fx en Facebook-gruppe.

For at undgå falske opslag og dårligt indhold har vi også bygget et admin-system ind, hvor nye opslag skal godkendes, før de bliver synlige for andre brugere. Det giver platformen et ekstra lag af troværdighed og sikkerhed.

Kravspecifikation

Den originale kravspecifikation blev udarbejdet i starten af projektforsløbet og beskrev de krav vi satte til systemet inden vi begyndte at bygge det. Herunder følger en opsummering af de centrale krav samt en beskrivelse af de ændringer vi har foretaget undervejs.

Oprindelige krav

Kravspecifikationen beskrev et system med følgende kernefunktioner: brugere skal kunne oprette en konto og logge ind med to roller, bruger og administrator. Brugere skal kunne oprette opslag for genstande de vil udleje, og nye opslag skal have status "Afventer" indtil en administrator godkender dem, kun godkendte opslag vises i browse-sektionen. Brugere skal kunne browse og søge i opslag med filtrering på kategori, titel og afstand. En bruger skal kunne sende en lejeforespørgsel med start- og slutdato, og systemet skal gemme status på forespørgslen. Der skal være en chat så bruger og udlejer kan aftale praktiske detaljer. Brugeren skal have et dashboard med overblik over egne opslag, forespørgsler og aktive lejemaal med afleveringsdato. Admin skal have et panel med overblik over alle brugere, opslag og lejeforespørgsler i systemet.

Følgende funktioner var markeret som optionelle i kravspecifikationen og ville kun blive implementeret hvis der var tid: kortvisning med interaktivt kort, integration med en rigtig betalingsløsning, ratings og anmeldelser samt rapportering af mistænkelige opslag.

Ændringer og udvidelser

Undervejs i udviklingen endte systemet med at blive betydeligt større end kravspecifikationen lagde op til. Det skete fordi vi efterhånden opdagede at visse funktioner var nødvendige for at systemet giver mening i praksis, fx giver det ikke mening at have udlån mellem fremmede uden et system der håndterer konflikter og skaber troværdighed.

Nedenstående funktioner var beskrevet i den originale kravspecifikation inden udviklingen begyndte. MK betyder minimumskrav, funktioner der skal være med. K betyder krav, funktioner der skal være med hvis muligt. O betyder optionel, funktioner der kun ville komme med hvis der var tid.

Funktion	Prioritet	Status
Opret konto og login med roller (User/Admin)	MK	Implementeret
Browse godkendte opslag	MK	Implementeret
Søgning og filtrering på kategori, title og afstand	MK	Implementeret
Opret opslag med titel, beskrivelse, kategori, pris og lokation	MK	Implementeret
Admin godkendelse af opslag (Pending > Approved/ Rejected)	MK	Implementeret
Lejeforespørgsel med start og slutdato og statusflow	MK	Implementeret
Chat mellem bruger og udlejer	MK	Implementeret
Bruger dashboard med opslag, forespørgsler og aktive lejemål	MK	Implementeret
Admin overblik over alle brugere, opslag og lejeaftaler	MK	Implementeret
Rediger og slet egne opslag	K	Implementeret
Aktiv leje oprettes når forespørgsel accepteres	K	Implementeret
Billed upload på opslag	K	Implementeret
Notifikationer ved godkendelse, ny besked og accepteret forespørgsel	K	Implementeret
Validering af input i frontend og backend	K	Implementeret
Kortvisning med pins ud fra opslags lokation	O	Implementeret
Rating og anmeldelser efter endt leje	O	Implementeret
Rapporter mistænkelige opslag og bruger	O	Implementeret
Rigtig betalingsintegration (Stripe/ MobilePay)	O	Ikke Implementeret
Automatiske påmindelser via email/ push	O	Ikke Implementeret

Nye funktioner tilføjet undervejs

Disse funktioner stod ikke i den originale kravspecifikation. De blev tilføjet undervejs fordi vi vurderede at de var nødvendige for at systemet fungerer i praksis, eller fordi de naturligt fulgte af andre funktioner vi allerede byggede.

Funktion	Begrundelse
Brugerscore og troværdighedssystem	Et system til private udlån kræver en måde at vurdere brugerens pålidelighed. Scoren styrer også om en lejeforespørgsel kræver admin-godkendelse.
Twistesystem med bevisbilleder	Kravspecifikation nævnte konflikthåndtering som fremtidig udvidelse. Vi vurderede at det var nødvendigt for at platformen kan fungere trygt.
Bødesystem	Følger naturligt af klagesystemet, admin skal kunne pålægge bøder som resultat af en tvist eller manuelt.
Klagesystem	Brugere der er uenige i en adminafgørelse skal have mulighed for at klage den.
Identitetsverifikation med ID-upload	Verificerede brugere giver andre mere tryghed ved udlån og giver en scorebonus.
Support	Tilføjet så brugere har en direkte kanal til at få hjælp fra admin inde i systemet.
Brugerblokering	Brugere skal kunne blokere folk de ikke vil have kontakt med.
Direkte beskeder	Kravspecifikation beskrev kun lånechat. Direkte beskeder giver brugere mulighed for at kommunikere frit uden om en aktiv leje.
Leje-forlængelse	En låntager skal kunne anmode om ekstra dage, som udlejereren kan acceptere eller afvise.
Seneste sete genstande	Giver brugeren et hurtigt overblik over hvad de har kigget på for nylig.
QR-kode til afhentning og aflevering	Bekræfter præcist hvornår en genstand er afhentet og afleveret, og dokumenterer genstandens stand ved leje-start via et automatisk snapshot.
Email bekræftelse ved oprettelse	Brugere skal bekræfte deres email-adresse før kontoen er fuldt aktiv.
Online status	Systemet viser om en bruger er aktiv i øjeblikket, så man kan se om en udlejer er tilgængelig.
Admin broadcast notifikationer	Admin kan sende en notifikation til alle brugere på en gang, fx ved systemmeddelelser.

Definition af produktet

RentIt er en webapplikation der gør det muligt for private at leje og udleje genstande til hinanden. Ideen bag projektet er enkel: mange mennesker har ting stående derhjemme som de ikke bruger ret tit, og andre har brug for de samme ting, men vil ikke købe dem til fuld pris. RentIt er broen imellem dem.

I modsætning til eksisterende udlejningsvirksomheder, som ofte har et begrænset udvalg og tager høje gebyrer, er RentIt bygget til at almindelige mennesker selv kan bestemme hvad de vil udleje og til hvilken pris. Det kan være alt fra værktøj og havemaskiner til bøger, spil og kameragrej.

Systemet er bygget som en Single Page Application (SPA), hvor Angular håndterer frontend og ASP.NET Core Web API håndterer backend. Al data gemmes i en SQL Server database, og backend samt database kan startes op samlet med Docker Compose, mens frontend startes separat med npm install og ng serve.

Der er to brugerroller i systemet: en almindelig bruger og en administrator. Brugeren kan oprette opslag, browse efter genstande og sende lejeforespørgsler. Administratoren godkender nye opslag inden de bliver synlige for andre, og har overblik over alle brugere og aktivitet i systemet.

Funktionalitet

RentIt er bygget op omkring en række kernefunktioner som tilsammen udgør et komplet udlejningsflow fra start til slut. Systemet har undervejs i udviklingen fået betydeligt flere funktioner end den originale kravspecifikation beskrev.

Brugerhåndtering

Alle der vil bruge platformen skal oprette en konto med navn, email, adresse og adgangskode. Systemet gemmer også brugerens lokation så afstandsfiltrering fungerer korrekt. Når kontoen er oprettet kan brugeren logge ind og ud, og systemet husker hvem man er via JWT tokens med refresh token support, så man ikke bliver logget ud unødvendigt. Der er to roller i systemet, bruger og administrator, og adgangen til forskellige dele af systemet styres ud fra hvilken rolle man har.

Brugerscore og troværdighedssystem

Alle brugere starter med 100 point. Scoren stiger ved fx til-time-afleveringer og falder ved forsinkede afleveringer, skader og aflysninger. Scoren afgør hvilket adgangsniveau brugeren har til at låne genstande, brugere med en score over 50 kan låne direkte, brugere med en score mellem 20 og 50 kræver admin-godkendelse inden udlejeren ser forespørgslen, og brugere der falder under 20 point bliver blokeret fra at låne. Al scorehistorik gemmes så brugeren og admin altid kan se hvad der har påvirket scoren.

Opslag og udlejning

En bruger kan oprette et opslag for en genstand de vil udleje. Opslaget indeholder titel, beskrivelse, kategori, stand (Excellent/Good/Fair/Poor), pris pr. dag, depositum, lokation og billeder. Når et opslag er oprettet får det automatisk status "Afventer", og det bliver ikke vist offentligt før en administrator har godkendt det. Brugeren kan efterfølgende redigere eller slette egne opslag, og hvis et opslag afvises af admin kan brugeren rette det og sende det til godkendelse igen.

Browse og søgning

Brugere kan browse alle godkendte opslag og filtrere på kategori, titel, pris, stand og tilgængelighed. Der er også en afstandsfiltrering baseret på brugerens lokation, så man kan finde genstande tæt på sig selv. Systemet understøtter paginering så browse-siden forbliver hurtig selv ved mange opslag.

Favoritter og senest set

Brugere kan markere opslag som favoritter og nemt finde dem igen. Systemet holder også automatisk styr på hvilke opslag brugeren senest har kigget på, så det er nemt at vende tilbage til noget man var interesseret i.

Lejeforespørgsel og lånehåndtering

Når en bruger finder en genstand de vil leje, kan de sende en lejeforespørgsel med en start- og slutdato. Afhængigt af brugerens score kan forespørgslen enten gå direkte til udlejeren eller først til admin for godkendelse. Forespørgslen gennemgår en række statusser: Afventer, Admin-afventer, Godkendt, Aktiv, Gennemført, Forsinket, Afvist og Annulleret. Når en leje er aktiv kan låntager anmode om en forlængelse af lejeperioden, som udlejeren kan acceptere eller afvise.

Automatiske baggrundstjenester

Systemet kører en række automatiske processer i baggrunden uden at nogen skal gøre noget manuelt. Det inkluderer automatisk markering af forsinkede lån, automatisk lukning af inaktive supporttråde, automatisk udløb af afventende lejeforespørgsler og automatisk ophævelse af midlertidige brugerbans.

Chat og beskeder

Systemet har to former for kommunikation. Lånechat er knyttet direkte til en lejeaftale og bruges til at koordinere afhentning og detaljer. Direkte beskeder giver brugere mulighed for at chatte frit med hinanden. Begge chat-typer er bygget med SignalR så beskeder modtages i realtid uden at siden genindlæses.

Anmeldelser

Når en leje er gennemført kan begge parter anmelde hinanden og den udlejede genstand. Brugeranmeldelser giver en vurdering af udlejeren eller lejeren som andre brugere kan se, og anmeldelser af genstande hjælper andre med at vurdere om en genstand er det værd.

Twister og konfliktløsning

Hvis en genstand kommer tilbage beskadiget eller i en anden stand end aftalt, kan enten udlejer eller låntager åbne en tvist. Begge parter kan uploade bevisbilleder. Admin gennemgår tvisten og afgiver en dom der kan resultere i bøder eller scoreændringer for en eller begge parter. Er man uenig i afgørelsen kan man anke den.

Bødesystem

Admin kan udstede bøder til brugere, enten som resultat af en tvist eller manuelt. Brugeren kan indsende bevis for betaling, som admin herefter godkender eller afviser.

Rapportering

Brugere kan rapportere mistænkelige opslag, brugere, anmeldelser eller beskeder. Admin håndterer rapporterne og kan tage action. Brugere kan også blokere andre brugere de ikke vil have kontakt med.

Verifikation

Brugere kan ansøge om at få verificeret deres identitet ved at uploade et officielt ID-dokument som pas eller kørekort. Verificerede brugere får et badge og en scorebonus, hvilket giver andre brugere mere tillid til dem.

Support

Brugere kan oprette supporttråde direkte i systemet hvis de har problemer. Admin kan gøre krav på en tråd og håndtere den. Supporttråde lukkes automatisk hvis de er inaktive i et stykke tid.

Notifikationer

Systemet sender realtidsnotifikationer via SignalR til brugerne ved alle vigtige hændelser, fx når en lejeforespørgsel accepteres, når en genstand godkendes, når en tvist afgøres eller når man modtager en besked.

Dashboard

Hver bruger har et personligt dashboard hvor de kan se deres egne opslag med status, deres lejeforespørgsler, aktive lån med afleveringsdato, favoritter, senest sete genstande og deres scorehistorik.

Adminpanel

Administratoren har et separat panel med fuld kontrol over systemet. Admin kan godkende eller afvise opslag, håndtere lejeforespørgsler, se og administrere alle brugere, udstede bøder, afgøre tvister, behandle ankesager, håndtere rapporter, verificere brugere og besvare supporthenvendelser.

QR-kode bekræftelse og snapshot

Hvert opslag får automatisk genereret en unik QR-kode når det oprettes. QR-koden er kun synlig for udlejeren og admin. Når en leje starter, scanner enten udlejeren eller låntager QR-koden for at bekræfte at genstanden er afhentet, det sætter lejen til aktiv og registrerer afhentningstidspunktet. På samme måde bekræftes afleveringen ved at scanne QR-koden igen, og systemet registrerer hvornår genstanden kom tilbage. På det tidspunkt hvor lejen aktiveres, tager systemet automatisk et snapshot af genstandens billeder og stand, så der er dokumentation for genstandens tilstand ved leje-start. Det snapshot bruges som reference hvis der senere opstår en tvist.

Kortvisning

Systemet har en kortvisning hvor alle godkendte genstande vises som pins på et interaktivt kort. Kortet er bygget med Leaflet og bruger Geoapify til adressesøgning og geokodning. Brugeren kan søge efter en adresse eller et område, og kortet opdaterer sig med de genstande der er i nærheden. Man kan klikke på en pin for at se grundinfo om genstanden og gå direkte til opslaget. Kortvisningen er et alternativ til browse-siden og giver et visuelt overblik over hvad der er tilgængeligt i et bestemt område.

Begrænsning

RentIt er udviklet som en MVP (Minimum Viable Product), hvilket betyder at vi har fokuseret på de vigtigste funktioner frem for at bygge alt på en gang. Der er derfor nogle ting som systemet ikke håndterer i denne version.

Betaling

Systemet understøtter ikke rigtig online betaling i denne version. Lejepris og depositum bliver registreret i systemet, og brugerne kan angive betalingsmetode som MobilePay, kort, bankoverførsel eller kontant, men selve betalingen foregår mellem brugerne uden om platformen. En fremtidig version kunne integrere en betalingsgateway som fx Stripe eller MobilePay direkte i appen.

Forsikring

Systemet håndterer ikke forsikring ved skader. Vi har implementeret et tvistesystem der hjælper med at løse konflikter, men der er ingen egentlig forsikringsordning der automatisk dækker beskadigede genstande.

Levering og fragt

Systemet er bygget ud fra at brugerne selv aftaler afhentning og aflevering fysisk. Der er ingen integration med fragtservices eller leveringsløsninger.

Rate limiting

Systemet har ikke implementeret rate limiting på login-endpoints. Det betyder at der i teorien kan laves mange loginforsøg i træk uden at blive blokeret. I en produktionsversion ville man tilføje en mekanisme der låser en bruger ude midlertidigt efter et antal fejlede forsøg, for at reducere risikoen for brute-force angreb.

Email og push notifikationer

Systemet sender realtidsnotifikationer via SignalR mens brugeren er logget ind, men der er ikke implementeret email eller push notifikationer der kan nå brugeren når de ikke er aktive i systemet.

Fremtid - mulige udvidelser

Når systemet er færdigt som MVP, kan det udvides på flere måder. De mest oplagte udvidelser er en rigtig betalingsintegration med fx Stripe og en mere avanceret ID-verificering med fx MitID. Derudover kunne man på sigt udvide adminpanelet med automatiske regler for flagning af mistænkelige opslag og bedre logging og overvågning af systemet

Testkonditioner

For at sikre at RentIt fungerer som forventet har vi testet systemet på flere niveauer. Testene dækker forretningslogik i service-laget, repository-laget mod en in-memory database og controller-laget med mockede services.

Udviklingsmiljø

Systemet er primært testet lokalt på vores egne computere. Backend og database køres via Docker Compose, mens frontend startes separat med npm install og ng serve. Det betyder at hvert lag kan testes isoleret uden at hele systemet behøver at køre.

Unit tests - Service-laget

Vi har skrevet unit tests i xUnit der tester forretningslogikken i service-laget med Moq til at mocke alle eksterne afhængigheder. Disse tests kører uden database eller UI.

AuthService-tests dækker hele registrerings- og loginflowet. Vi tester at registrering fejler hvis emailen allerede er i brug, hvis brugernavnet er taget, og hvis adgangskoden er for svag. Vi tester at registrering sender en bekræftelsesemail og returnerer det korrekte svar. For login tester vi at et slettet, permanent bannet og midlertidigt bannet login alle fejler med de rette fejlbeskeder, og at en ubekræftet email blokerer login. Vi tester også at logout rydder refresh token korrekt, at glemt-adgangskode sender en reset-email, og at email- og brugernavnstjek returnerer korrekte svar.

AdminUserService-tests dækker brugeradministration fra admin-siden. Vi tester at opslag af en bruger returnerer korrekte data inklusiv udestående bøder, at forsøg på at hente en ikke-eksisterende bruger kaster en fejl, at en bruger kan soft-slettes når der ingen aktive lån eller tvister er, at en ban sender en notifikation til brugeren, og at en admin ikke kan banne en anden admin.

Unit tests - Repository-laget

Repository-testene bruger en in-memory SQLite database og tester at datahentning og -skrivning fungerer korrekt end-to-end mod en rigtig databasestruktur.

LoanRepository-testene dækker oprettelse og hentning af leje, hentning af aktiv leje pr. genstand, tjek for overlappende datoer, filtrering på status og tjek for løbende lån pr. bruger.

ItemRepository-testene dækker oprettelse og hentning af opslag, tjek for eksisterende slug og QR-kode, ejertjek, antal afventende godkendelser og antal tilgængelige opslag i systemet.

DisputeRepository-testene dækker oprettelse og hentning af tvist, aktiv tvist pr. leje, antal åbne tvister og sletning af bevisfoto.

FineRepository-testene dækker oprettelse og hentning af bøde, tjek for udestående bøder, summering af ubetalte beløb pr. bruger og antal afventende betalingsbeviser.

AppealRepository-testene dækker hentning af anke med tilknyttede detaljer, afventende anker pr. bruger, tjek for aktiv score-anke og hentning af bøde-anke pr. bøde.

SupportRepository-testene dækker oprettelse af supporttråd, tjek for aktiv tråd pr. bruger, hentning af ulæste beskeder og tilføjelse af ny besked.

NotificationRepository-testene dækker oprettelse, hentning pr. bruger og sletning af notifikationer.

UserBlockRepository-testene dækker blokering af bruger, tjek for blokering i begge retninger, hentning af blokerede bruger-id'er og fjernelse af blokering.

UserFavoriteRepository-testene dækker tilføjelse af favorit, tjek for om favorit allerede eksisterer, hentning af brugere der skal notificeres ved tilgængelighed og fjernelse af favorit.

UserReviewRepository-testene dækker oprettelse af anmeldelse, tjek for om en bruger allerede har anmeldt en anden og beregning af gennemsnitsvurdering.

ItemReviewRepository-testene dækker oprettelse og hentning af genstandsanmeldelse, hentning pr. leje, hentning af vurderinger pr. genstand og soft-sletning af anmeldelser.

DirectConversationRepository-testene dækker hentning af samtale uanset hvilken retning brugerne er angivet i, oprettelse af samtale med konsistent rækkefølge, hentning af blokerede bruger-id'er og tæller for ulæste beskeder.

LoanMessageRepository-testene dækker oprettelse af besked, hentning af seneste besked og hentning af ulæste beskeder pr. bruger.

UserRecentlyViewedRepository-testene dækker tilføjelse af senest set, hentning, antal pr. bruger og sletning af den ældste post når grænsen er nået.

UserBanHistoryRepository-testene dækker oprettelse af ban-post og hentning af den seneste ban pr. bruger.

ScoreHistoryRepository-testene dækker oprettelse af scorepost, hentning pr. bruger og summering af scoreændringer.

ReportRepository-testene dækker oprettelse af rapport, tjek for om en bruger allerede har rapporteret samme mål og hentning af seneste rapporteringstidspunkt.

VerificationRequestRepository-testene dækker oprettelse af ansøgning, tjek for afventende ansøgning og hentning af afventende ansøgning pr. bruger.

Unit tests - Controller-laget

Controller-testene bruger mockede services og tester at HTTP-laget kalder de rette servicemetoder med de rette parametre og returnerer de korrekte HTTP-svar.

AuthController-tests dækker at registrering returnerer 200 med en bekræftelsesbesked, at email-bekræftelse returnerer 200 eller 400 afhængigt af resultatet, at login returnerer et token, at refresh og logout virker korrekt, og at glemt-adgangskode altid returnerer 200.

ItemController-tests dækker at browse-endpoints sender filter og paginering videre korrekt, at oprettelse returnerer 201 med den rigtige route, at opdatering og sletning sender det rigtige bruger-id videre, og at QR-kode, foto-upload, slet foto og sæt primært foto alle kalder de rette servicemetoder.

CategoryController-tests dækker at anonyme brugere, almindelige brugere og admin-brugere alle ser den rette mængde kategorier, og at opret, opdater, toggle og slet alle fungerer.

En samlet oversigt over testresultater findes i Bilag 7 - Testrapport.

Brugertest

Vi har lavet en kort brugertest med 3-5 personer fra vores klasse, hvor de fik konkrete opgaver de skulle løse i systemet. Vi observerede hvor de blev forvirrede og hvad der fungerede godt, og brugte resultaterne til at forbedre brugergrænsefladen efterfølgende. Opgaverne var at oprette en bruger og logge ind, oprette et opslag, finde en genstand i browse og sende en lejeforespørgsel, og finde mine lejemål og se afleveringsdatoen.

Brugervejledning

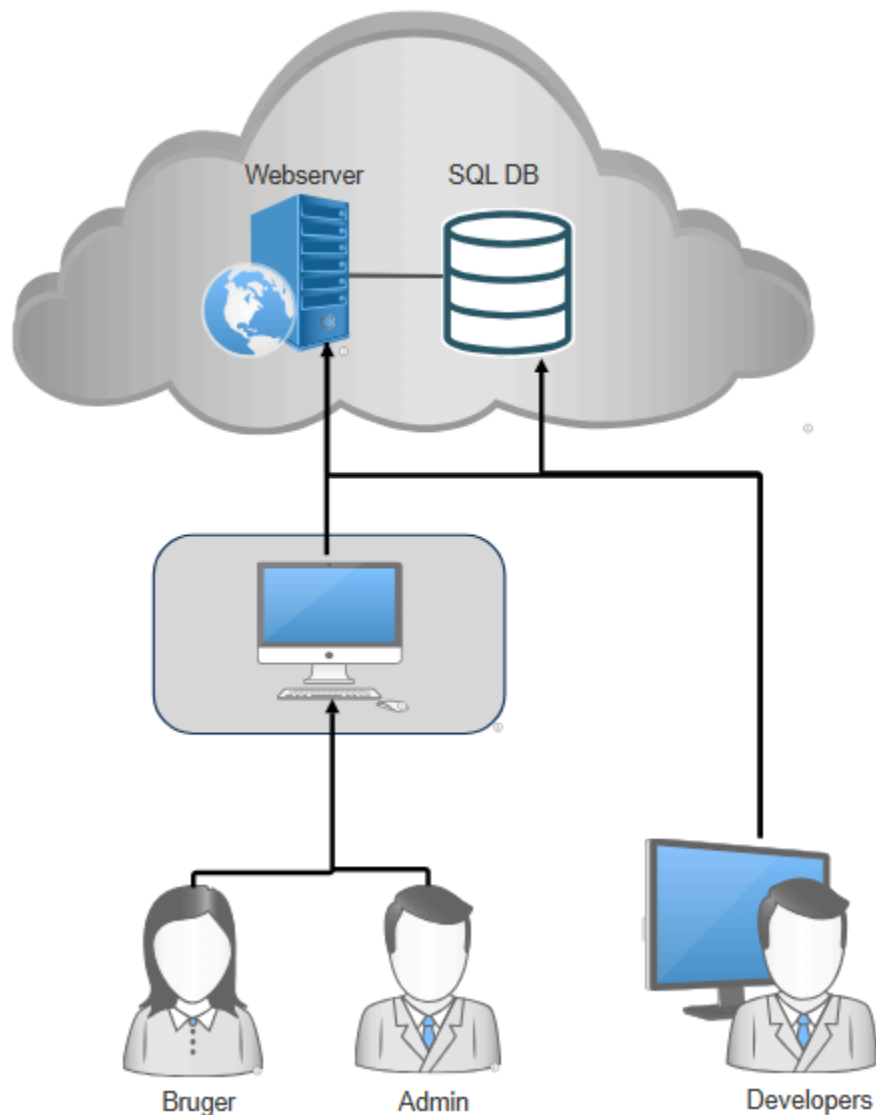
En detaljeret brugervejledning findes i Bilag 8

Teknisk produktdokumentation

Systemsarkitektur

RentIt er bygget som en tre-lags arkitektur bestående af en frontend, en backend og en database. Backend og database kommunikerer med frontend via et REST API. Backend og database køres samlet med Docker Compose, mens frontend startes separat. Figuren nedenfor illustrerer hvordan systemet hænger sammen.

Aktør kontekst diagram



Figur 1: Aktør kontekst diagram

Aktør beskrivelse

Aktør navn	Bruger
Type	Primær aktør
Beskrivelse	Brugeren er den almindelige bruger af systemet og kan både være lejer og udlejer. Brugeren kan oprette en konto, logge ind og oprette opslag for ting de vil udleje med pris pr. dag og depositum. Brugeren kan browse andre opslag og filtrere på kategori, pris, stand og afstand. Når brugeren finder noget de vil leje, sender de en lejeforespørgsel. Afhentning og aflevering bekræftes ved at scanne en QR-kode. Brugeren har et dashboard med overblik over egne opslag, forespørgsler, aktive lejemål med afleveringsdato, favoritter, senest sete genstande og scorehistorik. Brugeren har en score der påvirker deres adgang til at låne genstande. De kan ansøge om identitetsverifikation, åbne tvister, anke afgørelser, rapportere andre brugere, blokere brugere og oprette supporttråde.
Forventet teknisk niveau	Lavt til middel

Aktør navn	Administrator (Admin)
Type	Primær aktør
Beskrivelse	Administratoren er systemets moderator og har ansvar for at platformen fungerer sikkert og at indholdet er gyldigt. Admin godkender eller afviser nye opslag og lejeforespørgsler fra brugere med lav score. Admin håndterer tvister og afgiver domme der kan resultere i bøder og scoreændringer. Admin behandler ankesager, verificerer brugeridentiteter, håndterer rapporter og besvarer suppothenvendelser. Admin kan manuelt justere brugeres score, udstede bøder, banne brugere og sende notifikationer til alle brugere på en gang.
Forventet teknisk niveau	Middel

Aktør navn	Developers
Type	Sekundær aktør
Beskrivelse	Developers er dem der udvikler, vedligeholder og drifter løsningen. De har adgang til servermiljøet (webserver og database) for at kunne opsætte systemet, deploye nye versioner og overvåge stabilitet. Udviklere arbejder med fejlretning, forbedringer og opdateringer af både frontend og backend, samt database-migreringer og backup. De sørger også for sikkerhed i systemet, fx korrekt håndtering af login (hashing af passwords), API-sikkerhed, roller/rettigheder og at kommunikationen kører over HTTPS/SSL. Developers er ikke en almindelig brugerrolle i selve app'en, men en teknisk aktør der sikrer at systemet fungerer og kan videreudvikles.
Forventet teknisk niveau	Højt

Frontend

Frontend er bygget i Angular og fungerer som en Single Page Application (SPA). Det betyder at siden ikke genindlæser sig selv når man navigerer rundt, hvilket giver en hurtigere og mere flydende brugeroplevelse. Frontend kommunikerer med backend udelukkende via HTTP requests til REST API'et, og al præsentationslogik ligger her, fx visning af opslag, formularer og dashboard.

Backend

Backend er bygget i ASP.NET Core Web API med C#. Det er her al forretningslogikken ligger, fx godkendelse af opslag, håndtering af lejeforespørgsler og rollestyring. Backend eksponerer en række endpoints som frontend kalder, og alle endpoints er dokumenteret og kan testes via Swagger. Backend bruger Entity Framework Core som ORM til at kommunikere med databasen, hvilket betyder at vi arbejder med databasen gennem C# objekter frem for rå SQL.

Database

Databasen er en SQL Server database der gemmer alle data i systemet, brugere, opslag, lejeforespørgsler, lejeaftaler og chatbeskeder. Databasen tilgås kun gennem backend, aldrig direkte fra frontend, hvilket er med til at sikre at data ikke kan tilgås eller manipuleres uden om vores forretningslogik og sikkerhedstjek.

Docker Compose

Backend og database er sat op med Docker Compose, så API'et og SQL Server-databasen kan startes samlet med en enkelt kommando. Det gør det nemt at køre backend lokalt og sikrer at alle gruppemedlemmer arbejder i det samme miljø uanset hvilken computer de bruger. Frontend køres separat, man installerer dependencies med npm install og starter udviklingsserveren med ng serve. Det betyder at frontend og backend kan startes og stoppes uafhængigt af hinanden under udvikling, hvilket gør det hurtigere at arbejde på et lag ad gangen.

SignalR og realtidskommunikation

Realtidskommunikation er implementeret med SignalR, som opretter og vedligeholder en vedvarende forbindelse mellem frontend og backend. Det bruges til tre ting: chat, notifikationer og online-status. Der er fire SignalR hubs i systemet, en til lånechat, en til direkte beskeder, en til notifikationer og en til supportchat. Det betyder at beskeder og notifikationer leveres øjeblikkeligt uden at siden skal genindlæses. Systemet tracker også om brugere er online via en separat hub, så andre brugere kan se om en udlejer er aktiv i øjeblikket.

Billedopbevaring

Billeder uploades og opbevares via Cloudinary, som er en ekstern cloud-tjeneste til mediehandling. Når en bruger uploader billeder til et opslag eller et ID-dokument til verifikation, sendes billederne direkte til Cloudinary, og systemet gemmer kun URL'erne i databasen. Det betyder at billederne serveres hurtigt via Cloudinary's CDN og at vi ikke belaster vores egen server med at opbevare store filer.

Kodestruktur

Backend er struktureret i tydelige lag med controllers, services, repositories, interfaces og DTO'er. Controllers håndterer HTTP-requests og kalder services. Services indeholder al forretningslogik og kalder repositories for at tilgå databasen. Interfaces bruges til at løskoble lagene fra hinanden, så services kan testes uden at databasen er tilgængelig. DTO'er bruges til at styre præcis hvilke data der sendes til og fra API'et, så interne databasemodeller ikke eksponeres direkte. Frontend er struktureret med Angular services der håndterer al kommunikation med API'et, og komponenter der kun håndterer visning og brugerinteraktion.

Kommunikationsflow

Når en bruger fx sender en lejeforespørgsel, sker følgende: Angular sender et HTTP request til backend API'et med de nødvendige data. Backend validerer requestet, tjekker at brugeren er logget ind og har de rette rettigheder via JWT token, udfører forretningslogikken og gemmer data i databasen via Entity Framework. Til sidst sender backend et svar tilbage til Angular, som opdaterer brugergrænsefladen uden at genindlæse siden. For realtidshændelser som chatbeskeder og notifikationer bruges SignalR i stedet for HTTP. Her opretholder frontend en vedvarende forbindelse til backend, og serveren kan sende data direkte til klienten uden at klienten behøver at spørge først.

Sikkerhed

Sikkerhed har været et vigtigt fokuspunkt i udviklingen af RentIt. Vi har implementeret følgende sikkerhedsforanstaltninger:

Password hashing Vi bruger ASP.NET Identity til at håndtere brugere, hvilket betyder at adgangskoder aldrig gemmes i klar tekst i databasen. De hashes automatisk før de gemmes, så selv hvis nogen fik adgang til databasen ville de ikke kunne læse passwords direkte.

JWT tokens med refresh tokens Når en bruger logger ind udstedes et JWT token samt et refresh token. JWT tokenet sendes med ved alle efterfølgende requests til API'et og sikrer at kun

loggede ind brugere kan tilgå beskyttede endpoints. Refresh tokens sørger for at brugeren automatisk får et nyt token når det gamle udløber, så man ikke bliver logget ud midt i en session.

Rollebaseret adgang Systemet har to roller: User og Admin. Admin endpoints er beskyttet så en almindelig bruger får en 403 Forbidden fejl hvis de forsøger at tilgå dem. Dette gælder både i backend og i frontend hvor admin sider er skjult for brugere uden admin rollen.

Input validering Der er validering både i frontend og backend, så ugyldige værdier som fx en negativ pris eller en slutdato før startdatoen bliver afvist med en meningsfuld fejlbesked. Det sikrer at der ikke kan gemmes ugyldige data i systemet.

HTTPS I produktion skal systemet køre over HTTPS med et SSL certifikat så data mellem bruger og server er krypteret. I udvikling bruges et lokalt udviklingscertifikat.

Ikke-funktionelle Krav

De ikke-funktionelle krav beskriver hvordan systemet skal opføre sig og hvilke kvalitetskrav der er til det, altså ikke hvad det kan, men hvordan det gør det.

Responsivt design Systemet skal fungere på mobil, tablet og computer. Brugergrænsefladen er bygget responsivt så layoutet tilpasser sig skærmstørrelsen, og alle centrale funktioner skal være tilgængelige uanset hvilken enhed brugeren benytter.

Brugervenlighed UI skal være overskueligt og ensartet med samme tema og komponenter på tværs af alle sider. Brugeren skal kunne finde rundt uden at skulle læse en vejledning, og fejlbeskeder skal være forståelige og hjælpsomme frem for tekniske.

Skalerbarhed Koden er struktureret i tydelige lag med services, DTO'er og controllers, så systemet er nemt at udvide med nye funktioner uden at skulle omskrive store dele af kodebasen. API'et er bygget generisk nok til at det på sigt kan understøtte fx en mobilapp eller integration med tredjeparter.

Fejlhåndtering Systemet skal håndtere fejl på en pæn måde. Det betyder at ugyldigt input afvises med meningsfulde fejlbeskeder, og at systemet ikke crasher hvis noget går galt. Der er validering både i frontend og backend, så fejl fanges så tidligt som muligt.

Sikkerhed Brugerdata håndteres sikkert og adgang styres via roller. Passwords gemmes aldrig i klar tekst men hashes med ASP.NET Identity. Alle beskyttede endpoints kræver et gyldigt JWT token, og admin endpoints er kun tilgængelige for brugere med admin-rollen. I produktion skal systemet køre over HTTPS.

Databeskyttelse Brugerdata opbevares kun så længe det er nødvendigt og adgangen til databasen er begrænset til backend serveren. Der er ikke implementeret fuld GDPR-håndtering i MVP'en, men det er en naturlig udvidelse til en fremtidig version.

Ydeevne Systemet er bygget med asynkrone kald i backend så det kan håndtere flere samtidige brugere uden at blive langsomt. SPA-arkitekturen i frontend betyder også at brugeren oplever hurtige sideovergange da siden ikke genindlæser sig selv ved hver navigation.

Hardware/Software miljø

Systemet består af en frontend, en backend og en database. Backend og database kører i Docker containers og startes samlet med Docker Compose. Frontend køres separat ved at installere dependencies med npm install og starte udviklingsserveren med ng serve.

Teknologier

Teknologi	Anvendelse
Angular	Frontend
ASP.NET Core Web API	Backend
Entity Framework Core	ORM/ databaseadgang
SQL Server	Relationel database
ASP.NET Identity	Brugerhåndtering og password hashing
JWT	Token-Baseret authentication
Docker Compose	Kørsel af backend og database samlet
Swagger	Dokumentation og test af API endpoints
GitHub	Versionsstyring og samarbejde
SignalR	Realtidskommunikation - chat og notifikationer
Cloudinary	Billedopbevaring og CDN
Leaflet	Interaktivt kort i frontend
Geoapify	Adressesøgning og geokodning til kortvisning

Forkortelse og begreber

Nedenstående tabel forklarer de forkortelser og tekniske begreber der bruges i rapporten.

Forkortelse/ Begreber	Betydning
API	Application Programming Interface- regler for hvordan to systemer kommunikerer. Bruges mellem frontend og backend i RentIt.
CDN	Content Delivery Network- netværk af servere der leverer filer hurtigt baseret på geografisk placering. Bruges af Cloudinary.
CRUD	Create, Read, Update, Delete - de fire grundlæggende operationer man kan udføre på data i en database.
DTO	Data Transfer Object- objekt der flytter data mellem lag i systemet uden at eksponere interne databasemodeller direkte.
GDPR	General Data Protection Regulation- EU's forordning der regulerer indsamling og opbevaring af persondata.
HTTP/ HTTPS	Hypertext Transfer Protocol / Secure- protokol til kommunikation på internettet. HTTPS er den krypterede version.
JWT	JSON Web Token- token der udstedes ved login og beviser over for API'et at brugeren er logget ind med de rette rettigheder.
MVP	Minimum Viable Product - en version af produktet der indeholder de vigtigste funktioner, så det kan demonstreres og testes uden at alt er bygget.
ORM	Object-Relational Mapper- oversætter mellem databasetabeller og C# objekter. RentIt bruger Entity Framework Core.
REST	Representational State Transfer - en arkitekturstil for API'er der bruger standard HTTP-metoder som GET, POST, PUT og DELETE.
SignalR	Et bibliotek fra Microsoft der gør det muligt at sende data fra server til klient i realtid uden at klienten behøver at spørge først.
SPA	Single Page Application - en webapplikation der loader en gang og derefter opdaterer indholdet dynamisk uden at genindlæse siden.
SQL	Structured Query Language - det sprog der bruges til at arbejde med relationelle databaser.
SSL	Secure Sockets Layer - en krypteringsprotokol der sikrer at data mellem bruger og server ikke kan aflæses undervejs.
UI	User Interface - brugergrænsefladen, altså det brugeren ser og interagerer med i browseren.
xUnit	Et testframework til .NET der bruges til at skrive og køre tests af backend-logikken.

Bilagsoversigt

Nedenstående tabel giver et overblik over alle bilag tilknyttet RentIt-projektet. Bilagene er enten inkluderet direkte i rapporten, vedlagt som separate dokumenter eller placeret i procesrapporten.

Bemærk: Use case diagrammet findes i to versioner - det originale fra kravspecifikationen og en opdateret version. Diagrammet er opdateret fordi systemet undervejs udviklede sig betydeligt ud over den oprindelige kravspecifikation, og den opdaterede version afspejler de funktioner der faktisk er implementeret. Begge versioner er inkluderet i Bilag 1.

Bilag	Titel	Indhold	Placering
Bilag 1	Use case Diagram (Original og opdateret)	Det originale use case diagram fra kravspecifikationen samt den opdaterede version. Diagrammet er opdateret fordi systemet udviklede sig betydeligt ud over den oprindelige kravspecifikation. Begge versioner er inkluderet for at vise udviklingen.	<i>Separat dokument</i>
Bilag 2	Use case beskrivelser	Detaljerede beskrivelser af alle use cases inkl. aktør, forudsætninger, forventet forløb og alternative forløb.	<i>Separat dokument (samme som Bilag 1)</i>
Bilag 3	Aktør kontekst diagram	Diagram der viser systemets tre aktører - Bruger, Administrator og Developers og deres relation til webserveren og databasen.	<i>Produktrapport</i>
Bilag 4	Aktør beskrivelse	Beskrivelse af alle tre aktører med navn, type, beskrivelse og forventet teknisk niveau.	<i>Produktrapport</i>
Bilag 5	GANTT diagram	Estimeret tidsplan for projektforsløbet fra planlægning til aflevering, inkl. faser som implementering, UI, rapport og testing.	<i>Processrapport</i>
Bilag 6	ER Diagram	Entity-Relationship diagram over databasestrukturen med alle tabeller, relationer og nøgler.	<i>Separat dokument</i>
Bilag 7	Testrapport	Komplet testrapport med resultater fra 464 unit tests (controllers, services og repositories), manuelle Swagger API tests samt beskrivelse af brugertest og UI-test.	<i>Separat dokument</i>
Bilag 8	Brugervejledning	Vejledning til installation og anvendelse af RentIt, inkl. opstart med Docker Compose, opstart af frontend og gennemgang af de centrale brugerflows.	<i>Separat dokument</i>
Bilag 9	Logbog	Projektdagbog/logbog fra hver gruppemedlem gennem hele forløbet, samlet i et dokument	<i>Separat dokument</i>
Bilag 10	Kildekoden	Den fulde kildekode for både frontend og backend.	<i>Processrapport</i>